

Service Function Chain Embedding Meets Machine Learning: Deep Reinforcement Learning Approach

Yicen Liu^{id} and Junning Zhang^{id}

Abstract—With the emerge of the network function virtualization (NFV) and software-defined network (SDN), the SDN/NFV-enabled network has been recognized as one of the most promising technologies to efficiently achieve resource allocation for network service. By introducing the SDN/NFV technology, each service can be represented by a service function chain (SFC), which can deploy the virtualized network functions (VNFs) and chain them with corresponding flows allocation. Considering the dynamic and complex nature of mobile terminals in cloud networks, how to efficiently embedding SFCs remains as a challenging problem. However, the traditional methods (e.g., exact, heuristic, meta-heuristic, and game, etc.) are subjected to the complexity of cloud network scenarios with dynamic network states, high-speed computational requirements, and enormous service requests. Recent studies have shown that deep reinforcement learning (DRL) is a promising way to deal with the limitations of the traditional methods. However, DRL agent training easily suffers from the problem of slow convergence performance. In order to overcome this narrow, in this paper, we design a novel DRL framework based on the enhanced deep deterministic policy gradient (E-DDPG) for the efficient SFC embedding in the dynamic and complex cloud network scenarios. Simulation results validate the high efficiency of the proposed DRL framework as it not only converges faster than currently baseline algorithms, but also reduces the end-to-end delay down to at least 28.3% compared to the benchmarks. All our proposed algorithms and code are available at <https://github.com/jn-z/>.

Index Terms—SDN, NFV, SFC, dynamic embedding, DRL.

I. INTRODUCTION

WITH an ambitious increase in the volume of service demands over the past few years, more and more Internet of Things (IoT) terminals (e.g., mobile users or devices) are attempting to access their own services wherever and whenever via multiple interfaces, e.g., WiFi, NFC, LTE Bluetooth, etc. The traditional static data processing modes [1], [2], [3] (e.g., enhanced interior gateway, static service chain) are usually performed by manually crafting the

routing table entries. However, these modes are recognized to be complicated, slow, wasteful and highly error-prone, which seems hard to adapt to dynamic service requirements in the future cloud networks. That is to say, the fixed middlebox placement can not be optimal for all possible traffic patterns, which remarkably requires cumbersome and costly middlebox upgrades/relocation. In order to address the situations, the network framework design has been transiting from the monolithic pattern to the software paradigm [1], [3], and the software paradigm-based network architecture is gradually supported by some emerging and promising technologies, e.g., network function virtualization (NFV), software-defined networking (SDN) and mobile edge computing (MEC), etc. As a classical application, service function chain (SFC) [2] has been recognized as one of the most important technologies that can satisfy dynamic service demands in the SDN/NFV-enabled networks. SFC constructed with deployed virtualized network functions (VNFs) in order is a promising service paradigm.

The efficient and automatic SFC embedding for the SDN/NFV-enabled networks is becoming an active research area, which has been recognized as one of essential technologies in the field of SFC resource allocation (SFC-RA) [4], [5]. Researchers from the industry and academia circles have spared no effort to struggle for solving the problem at present. As for the industry, European Telecommunications Standards Institute (ETSI) has done a lot of works in the design of network architectures [3]. The other researchers in the academia circles propose efficient algorithms based on different network scenarios [2], [4], [5], which is also a focus of this paper.

When surveying the literature, we notice that existing optimization methods are mostly based on exact [6], [7], [8], heuristic [9], meta-heuristic [10], [11] and game [12], [13]. However, real-world networks have become more complicated and highly dynamic, making it hard to model, predict and control. In addition, traditional solutions heavily rely on prior knowledge and are usually designed in an offline manner, sacrificing the flexibility feature of NFV and SDN. Due to the essential characteristic of SFC, especially in the case of multi-hop transmission in “service function chain” (SFC) consisting of a set of ordered VNFs, it is hard to accurately model the SFC dynamic embedding problem. To make SFC embedding algorithm more practical, researchers try to seek an online flexible approach that can exploit the advantages of NFV and SDN, and can also be applied online to adapt to the

Manuscript received 3 June 2023; revised 27 October 2023 and 8 January 2024; accepted 10 January 2024. Date of publication 15 January 2024; date of current version 12 July 2024. This work was supported by the National Natural Science Foundation of China under Grant 62071483, 62201602, in part by the Research Plan Project of NUDT under Grant ZK22-27. The associate editor coordinating the review of this article and approving it for publication was J. J. Yang. (Corresponding author: Junning Zhang.)

Yicen Liu is with the National Key Laboratory of Blind Signal Processing, Chengdu 610041, China (e-mail: yicenliu@aeu.edu.cn).

Junning Zhang is with the National University of Defense Technology, Hefei 410073, China (e-mail: zjn20101796@sina.cn).

Digital Object Identifier 10.1109/TNSM.2024.3353808

complexity of cloud network scenarios with dynamic network states, high-speed computational requirements, and enormous service requests.

Recently, leveraging deep reinforcement learning (DRL) algorithms as an advanced tool to achieve SFCs embedding is a promising manner, which many studies have demonstrated [14], [15], [16]. However, it is well known that DRL algorithms (e.g., DQN, DDQN, DDPG and A3C) suffer from the “curse of dimension”. That is, training intelligent DRL agents would be extremely time consuming and resource consuming when the dimension of either environment state or control action is large. Furthermore, SFC embedding process involves complicated decisions with both discrete actions (i.e., VNFs placement) and continuous actions (i.e., flows scheduling) at the same time. This further hinders the convergence of agent training, and it is desirable to customize the DRL framework based on the network control problem characteristics. To this end, we are motivated to design an enhanced deep deterministic policy gradient (E-DDPG) framework for dealing with dynamic and online SFC embedding. Our proposal mainly reconstructs two buffer replays (i.e., diversity buffer replay and high-error buffer replay) based on the basic DDPG framework.

The contributions of this paper can be summarized as follows:

- The performance of the traditional methods (e.g., exact, heuristic, meta-heuristic, and game, etc.) for dealing with the SFC embedding might be weakened when applying to the dynamic conditions and the varying user traffic. We are motivated by the fact that the recent DRL-based techniques show some effectiveness in solving the SFC embedding problem towards SDN/NFV-enabled networks.
- The SFC embedding problem contains the complicated decisions with both discrete actions, i.e., VNF placement, and continuous actions, i.e., traffic flow scheduling. Thus, we formulate the SFC embedding problem as hybrid Markov decision process (MDP). Also, we detailly describe the necessary tuples that are required by hybrid MDP-based model.
- It is well known that the current DRL-based techniques (e.g., DQN, DDQN and A3C) easily suffer from the “curse of dimensionality”. In order to efficiently deal with this narrow, an Enhanced DDPG (E-DDPG)-based approach for the SFC embedding is proposed that not only significantly accelerates the agent training, but also reduces the end-to-end delays in edge clouds.
- The Google TensorFlow is used to achieve the proposed E-DDPG based algorithm for the SFC embedding problem. The optimal parameter combination of the proposed E-DDPG is selected via grid search. The simulation results show the outstanding performance of our proposal in terms of the convergence performance and the end-to-end delays, compared to the DDPG-based approach, the A3C-based approach, the DQN-based approach and the traditional heuristic-based approach.

The rest of our paper is organized in the following. Section II summarizes and analyzes the related works.

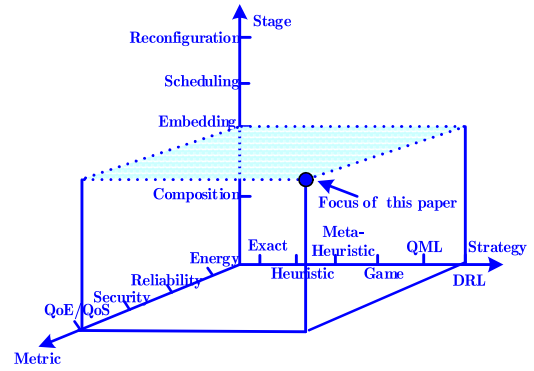


Fig. 1. Three-dimensional space regarding the SFC-RA.

Section III gives the system model and problem formulation. Next, our proposed approach for SFC embedding is presented in Section IV. Section V shows the simulation results and corresponding analysis. Finally, we conclude this paper in Section VI.

II. RELATED WORK

The SFC resource allocation problem (SFC-RA) has been investigated in the literature from various aspects including the system models, solutions and objectives. In the related work, we summarize and present the three-dimensional space in the regard of the SFC-RA, as shown in Fig. 1. To the best of our knowledge, current literatures in this regard are within the three-dimensional space. It is noted that this paper focuses on the SFC embedding problem, which is an important stage of SFC-RA. In Fig. 1, the shading plane represents the related works in the field of SFC embedding, and the point in the three-dimensional space is the focus in this paper.

In the following, we introduce the related work from three aspects, i.e., SFC-RA strategy, SFC-RA stage and SFC-RA metric.

A. SFC-RA Optimization Strategies

The χ -axis of the three-dimensional space represents the main categories of the strategies to solve the SFC-RA. As mentioned in Section I, such kind of SFC-RA is addressed by the exact, heuristic, meta-heuristic, game, DRL and quantum machine learning (QML). All literatures regarding SFC-RA are motivated by the fact that SFC-RA is an $\mathcal{NP}$ -hard problem and the $\mathcal{NP}$ -hard problem is hard to be solved in medium and large network scenarios. Six categories of SFC embedding approaches can be summarized as follows. **i) Mathematical optimization-based methods.** These approaches (e.g., integer linear program (ILP) [2], mixed integer linear program (MILP) [5], and integer non-linear program (INLP) [6]) are based on the mathematical optimization, which can be achieved by means of linear program algorithms. For example, Yang et al. [6] study the problem of how to place VNFs on edge clouds, and route the traffic among adjacent VNF pairs, and propose an exact integer non linear program. **ii) Heuristic-based methods.** The heuristics have been widely designed to solve the SFC embedding problem, since they have advantages

in terms of execution time. Some heuristic-based methods are Greedy with bias towards Least Load (GLL) [7], Greedy with bias towards Fast Processing (GFP) [8] and Viterbi [9]. For example, Nguyen et al. [17] propose a node ranking-based heuristic for SFC embedding, which has been proved that it is a good alternative in system where the network is not dynamic. **iii) Metaheuristic-based methods.** The modelling and embedding of SFC requests can be done from the evolutionary perspective, such as Genetic Algorithm (GA) [10], Tabu Search (TS) [7] and Variable Neighborhood Search (VNS) [11]. For example, Rankothge et al. [10] focus on optimizing the SFC-RA in the core cloud center, and propose a Genetic Algorithm (GA)-based method. **iv) Game-based methods.** Recent researchers attempt to adopt some game approaches to solve the SFC embedding problem [12], [13], such as segment auction scheme profit auction scheme and matching game. For example, Dieye et al. [13] propose a cross-domain SFC embedding approach based on the incomplete information game model on auction. However, the game model can only be addressed when simultaneously satisfying five tuples, i.e., player, action, strategy, information and payoff. **v) Quantum machine learning (QML) approach.** We have pioneered the use of QML into the SFC embedding, and propose two novel approaches, such as quantum evolution with feedback (QEF) [18] and fuzzy logic with quantum genetic algorithm (FQGA) [19]. As for the QML-based approaches, we have pioneered to apply the QML-based scheme to deal with SFC embedding at network edge [18], which uses the quantum code to efficiently iterate the solution, and adopt the fuzzy logic scheme to adaptively adjust the phase of the quantum rotation gate and mutation probability. **vi) Deep reinforcement learning (DRL) approach.** Recent achievements have demonstrated that DRL-based methods can effectively deal with SFC embedding in online and dynamic network scenarios. The popular DRL-based methods are deep Q network (DQN) [14], double deep Q network (DDQN) [15], and deep deterministic policy gradient (DDPG) [16], etc. For example, Pei et al. [15] formulate the SFC embedding as the DRL-based framework and propose a Double Deep Q Network (DDQN)-based SFC embedding algorithm in the core cloud networks. Fu et al. [20] propose a deep Q network (DQN)-based scheme with experience replay and target network, which can efficiently handle complex and dynamic SFC embedding.

B. SFC-RA Stages

In order to achieve the economies of scale expected from the SDN/NFV-enabled SFC technology, efficient usage of substrate resources should be acquired. There are four stages in the whole cycle of SFC-RA, and the -axis of the space indicates main stages of SFC-RA, including SFC composition, SFC embedding, SFC scheduling and SFC reconfiguration [21], [22], [23], [24]. **i) SFC composition.** This first challenge is the chaining process, that we call SFC composition. Network operator needs to efficiently compose such chains to deploy customized and dynamic SDN/NFV-enabled network services. To solve the SFC composition problem, Chemodanov et al. [25] provide a near-optimal

solution to coordinate the SFC composition for the geo-distributed cloud servers. This method admits QoS constraints such as latency, packet loss, etc. **ii) SFC embedding.** SFC embedding is the second challenge identified in the SFC-RA, which seeks to find where to allocate the VNFs in the network infrastructure in a suitable way and steer the traffic flow through orchestration order. As for SFC embedding problem, in our previous work [4], we consider different network topologies, and we devise two deep reinforcement learning (DRL)-based SFC embedding approaches for two network sizes: a DDPG based SFC embedding method for small-scale networks and an A3C based method for medium-scale networks. **iii) SFC scheduling.** A third stage of SFC-RA problem is the scheduling process. This stage attempts to give an answer to how to execute each function in order to minimize the total execution time without degrading the service performance, and respecting all the precedences and dependencies between the VNFs composing the network service. As for SFC scheduling problem, Bhamare et al. [26] address the problem of scheduling micro-services across the multiple cloud infrastructures and propose a fair weighted affinity-based scheduling heuristic to solve the SFC scheduling problem. **iv) SFC reconfiguration.** A final stage of SFC-RA problem is the reconfiguration process. This stage attempts to determine the migration of VNFs and re-routing service path when the network system reaches the reconfiguration trigger condition. To efficiently solve the SFC reconfiguration problem, Eramo et al. [27] have pioneered to study the SFC dynamic reconfiguration problem, and propose a policy that determines when and where to migrate the VNF instance and reroute the traffic flows in response to changes to the SFC request intensity.

C. SFC-RA Objectives and Metrics

The -axis of the space indicates the attribute requirements of SFC-RA, which varies from different applications of the use cases (e.g., IoT, ICN and ETSI) [24]. Solving SFC-RA requires a strategic objective to optimize. Objectives related with QoS/QoE, security, reliability/availability, profit maximization, fault tolerance, load balancing, energy saving, etc., are common examples of SFC-RA's main goals. The classical literatures regarding SFC embedding based on different service requirements can be summarized as follows. **i) QoS/QoE.** A network service that provides voice over IP services needs to count on low end-to-end delay requirements. In order to address the delay-aware SFC embedding problem, Tajiki et al. [28] study QoS-aware path allocation and VNF placement for the service function chain, and propose a single but effective heuristic consisting in iteratively deploying the VNFs in series using the nearest search procedure, which avoids exploring all possible solutions including some redundant ones. **ii) Reliability/Availability.** The failure of any single VNF causes the breakdown of the entire chain, which results in resource wastage, delays, and significant data loss. Therefore, a natural objective would be to maximize the reliability/availability of the whole chain. In order to reach this goal, Qu et al. [29] propose an incremental method that

TABLE I
SUMMARY OF THE CURRENT METHODS FOR SFC EMBEDDING

Strategy	Reference	Contribution/Objective	Shortcomings or Research Opportunities
Exact	[6]	Some formulate the SFC embedding optimization problem as an ILP, others as an MILP and others as an INLP, and these model can be solved by the exact solvers, e.g., GLPK, CPLEX.	The ILP-based solution is the optimal solution and requires extremely high computing resources, which cannot be applied in a real network context.
	[7]		
	[8]		
Heuristic	[7]	Most of the papers found in the literature fall into the category of heuristics-based solutions, which can divide the SFC embedding problem into some smaller subproblems and solve the optimal solution of the subproblems.	The heuristics generally require a total redesign of the algorithm. Moreover, the heuristics have a rapid convergence at the price of the risk of sticking at a local minimum that may not be effective.
	[8]		
	[9]		
	[42]		
Meta-heuristic	[7]	Some of the existing work falling in this category consider using evolutionary algorithms (e.g., GA, TS and SA) for the SFC embedding.	The performance of meta-heuristic-based approaches is mainly affected by some controlling parameters, and thus they can only perform well under certain conditions in the dynamics.
	[10]		
	[11]		
Game	[12]	Some attempt to adopt some game approaches to solve the SFC embedding problem, such as segment auction scheme, profit auction scheme and matching game.	The game-based problem can only be addressed when simultaneously meeting five tuples, i.e., player, action, strategy, information and payoff. However, it is hard to obtain five tuples at the same time.
	[13]		
	[21]		
Quantum machine learning	[18]	Our team has pioneered the use of QML into the SFC embedding, such as quantum evolution with feedback and fuzzy logic with quantum genetic algorithm.	The implementation of the QML-based approaches is currently impossible.
	[19]		
Deep reinforcement learning	[4]	Recent achievements have shown the potential of DRL approaches to solve the SFC embedding problem, e.g., deep Q network, double deep Q network, asynchronous advantage actor-critic, deep deterministic policy gradient.	The current DRL algorithms suffer from the “curse of dimension”. Moreover, the convergence performance of the current DRL-based approaches is slow.
	[14]		
	[15]		
	[16]		
	[20]		

can determine the number of the required VNF backups, thus achieving the availability-aware joint VNFs embedding and traffic flows. The authors use an availability-aware SFC resource allocation strategy that could significantly trade off the reliability, CPU and bandwidth consumption of a service chain. **iii) Security.** An important and promising application for the SDN/NFV-enabled SFC technology is in network security, where can dynamically and flexibly accomplish the chaining virtualized security network functions (VSNFs), e.g., network address translation, antispam and packet filter firewall, etc., and thus inspect, monitor or filter traffic flows in the cloud datacenter networks. For example, Shameli-Sendi et al. [30] propose a greedy heuristic based dynamic security service chain (SSC) orchestration approach, which selects the security function instance with the maximum network utility in an iterative cycle. **iv) Energy.** Due to the importance of the energy consumption for communication networks and its environmental effects, this objective is directly proportional to minimize the energy spent by the servers. To adapt to the different energy-aware service requirements, Xu et al. [31] define the energy aware VNF resource allocation problem, and carry out an evaluation of the power effectiveness of software data planes, virtual I/O, and software middleboxes. Lin et al. [32] consider a dynamic traffic scenario, and propose three polynomial heuristic algorithms for energy-aware SFC resource allocation. **v) Load Balancing.** Fei et al. [33] propose an architecture for distributed NFV deployment using central offices Selection (CS) and VNF Assignment (VA), in order to ensure load balancing and cost effectiveness of compute and bandwidth resources across all chosen COs. Wang et al. [34] focus on the multi-resource load balancing problem in NFV by first suggesting dominant load, or the load of the resource that

is under the most stress on a server, as the load balancing metric, and formulate the multi-resource load balancing problem as an optimization to reduce the maximum dominating load of all NFV servers.

D. Summary

Table I summarizes the most relevant paper related to SFC embedding in the SDN/NFV-enabled networks. In this paper, we consider the SFC embedding problem, and the objective function of this paper is the end-to-end delay, which is one of the most important metrics in the Service Level Agreement (SLA). As for the strategy selection, considering the complexity of the cloud network scenarios with dynamic network states, high-speed computational requirements, and enormous service requests, leveraging deep reinforcement learning (DRL) algorithms as an advanced tool to achieve SFCs embedding is a promising method. The DRL approach makes it possible to overcome the limits of scalability of the traditional approaches, by using a much more efficient approximation thanks to the use of neural networks. Inspired by the pioneering studies for DRL algorithms, we are motivated to design a novel DRL algorithm for SFC dynamic embedding. Moreover, in order to obtain a better efficiency, we propose several improvements to the existing DDPG technique, which can better explore the space of SFC embedding solutions.

III. SYSTEM MODEL

In order to formulate the SFC embedding problem, we model the substrate network and service function chain as an indirected graph and a directed graph, respectively.

Substrate Network: The cloud computing infrastructure consists of a set $\mathbb{N}$ of networked servers, and is represented as an indirected graph $\mathbb{G} = \{\mathbb{L}, \mathbb{N}, A_n, A_l\}$, where $\mathbb{N}$ is a set of substrate nodes, $\mathbb{L}$ is a set of links between the substrate nodes, A_n is the attributes of the substrate node n in $\mathbb{N}$, A_l is the attributes of the physical link $l \in \mathbb{L}$. More specifically, $\mathbb{N} = \mathbb{E} \cup \mathbb{Z} \cup \mathbb{C}$, where $\mathbb{E}$, $\mathbb{Z}$ and $\mathbb{C}$ denote the edge clouds, the switch/router nodes and the core clouds, respectively. Switch/router nodes are used to forward the traffic. The switch/router nodes can be placed in a transport network between edge and core clouds, and can be also deployed within edge and core cloud networks. Not only can the VNFs be deployed onto one or more cloud infrastructures, but also can be shared by different service demands. Each edge cloud $e \in \mathbb{E}$ builds a mount of edge servers along with the base station in an area, and provides the network service with end users $\mathbb{U}$. The end users are within the coverage of the base station. A user's service request correspondingly occurs when an arbitrary mobile user $q \in \mathbb{U}$ accesses the nearest base station. $A_n = \{\pi(n), T_n^v\}$, where $\pi(n)$ represents the total amount of the IT resources of the substrate node $n \in \mathbb{N}$, T_n^v represents the processing delay of each VNF $v \in \mathbb{F}$ on the substrate node $n \in \mathbb{N}$. $A_l = \{\rho(l), T_l\}$, where $\rho(l)$ is the transmission capacity on the physical link $l \in \mathbb{L}$, T_l indicates the flow delivering delay on the physical link $l \in \mathbb{L}$. Note that $l_{m,n} \in \mathbb{L}$ represents a link from the substrate node $m \in \mathbb{N}$ to $n \in \mathbb{N}$.

Service Function Chain: A service provider provides a set $\mathbb{R}$ of different network services by renting virtualized resources from the infrastructure provider, and a network service $r \in \mathbb{R}$ is provided in form of SFC as a set of ordered VNFs. Thus, SFC can be represented as $\mathbb{G}_r = (i, e, \mathbb{F}_r, \mathbb{S}_r)$, where $i \in \mathbb{N}$ and $e \in \mathbb{N}$ respectively denote the ingress node and egress node. $\mathbb{F}_r$ denotes the set of VNF required by service $r \in \mathbb{R}$ and $s_{v,u} \in \mathbb{S}_r$ enforces the flow traverse sequence between two consecutive VNFs $v, u \in \mathbb{F}_r$ along an SFC. To clearly tract the flow relationship between the servers for any SFC, a directed VNF graph $\mathbb{G}_v = (\mathbb{V}, \mathbb{S}_v)$ is built based on the SFC graph $\mathbb{G}_r$, as shown in Fig. 1. It is also called service logic of an SFC, where $\mathbb{V}$ represents the set of candidate VNF embedding state, $\mathbb{S}_v$ denotes the set of logical links. The VNF graph is built according to the following rules. $v_n^f \in \mathbb{V}$ is introduced to indicate that a candidate VNF $f \in \mathbb{F}_r$ is placed onto the substrate node $n \in \mathbb{N}$. For each network service $r \in \mathbb{R}$, strictly following the VNF sequence, a network flow processed by $v_n^{f'}$ shall be passed to the next VNFs in order. Thus, a directed link $s(v_n^{f'}, v_n^f) \in \mathbb{S}_v$ is added in set $\mathbb{S}_v$ for each $v_n^f \in \mathbb{V}$ whenever $s_{v,u} \in \mathbb{S}_r, \forall r \in \mathbb{R}$.

The delay-aware SFC embedding problem can be described as follows.

Definition 1: SFC Embedding Problem: Assume that there is a substrate network $\mathbb{G} = \{\mathbb{L}, \mathbb{N}, A_n, A_l\}$, and a set of new requests $\mathbb{R}$. For each request $r \in \mathbb{R}$, the SFC embedding problem is to focus on the scheme for chaining VNFs on cloud nodes, and steering the traffic through these VNFs in order before reaching the destination, such that the end-to-end delay is minimized and IT resources (i.e., CPU,

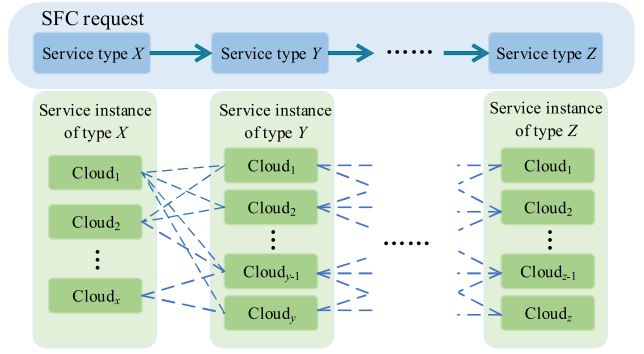


Fig. 2. Directed graph including SFC request and its corresponding candidate.

storage), bandwidth, latency and order constraints are also satisfied.

IV. ENHANCED DEEP DETERMINISTIC POLICY GRADIENTS (E-DDPG) BASED SFC EMBEDDING FRAMEWORK

With the success of DRL in online control optimization problem [35], [36], [37], we use the DRL technique to obtain near-optimal solution of dynamic and online SFC embedding. In this section, we formulate the SFC embedding as the HMDP model, and describe the tuples that are required by the DRL model. Next, we present the proposed E-DDPG algorithm.

A. Modelling With Hybrid Markov Decision Process (HMDP)

HMDP can determine the optimal decision in the stochastic dynamic systems. Thus, we formulate the SFC embedding as the HMDP model, and the elements of a DRL-based task can be defined as a three-tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{R} \rangle$ representing the state, action and reward, respectively. In our dynamic SFC embedding scheme based on DRL, at the time slot t , upon a state $\mathcal{S}(t)$, after taking action $\mathcal{A}(t)$, the agent shall observe a new state $\mathcal{S}(t+1)$, and the corresponding reward $\mathcal{R}(t)$, which can be used to evaluate the effectiveness of action.

We define the state, action and reward associated with the considered problem as follows:

1) **State:** It is assumed that the VNFs can provide sufficient resources. In order to decide the course of action to take for a particular VNF instance based on its present state, the central management node must consider the abundance and dynamics of states, including current flow rate and VNF placement states of all substrate nodes. Therefore, we represent the state space of the SFC embedding environment by the states of current flow rate and VNF placement. The state $\mathcal{S}(t)$ at time slot t is represented by a vector consisting of the flow rate of each SFC and the current VNF placement status, i.e.,

$$\mathcal{S}(t) = \langle T^r(t), x_n^f(t-1) \rangle, \quad \forall f \in \mathbb{F}_r, n \in \mathbb{N} \quad (1)$$

where $T^r(t)$ denotes the flow rate requesting the service $r \in \mathbb{R}$ at time slot t , $x_n^f(t)$ is the binary variable indicating whether a flow passes through a VNF $f \in \mathbb{F}_r$ hosted on the substrate node $n \in \mathbb{N}$ at time slot t . If a flow passes through a VNF

$f \in \mathbb{F}_r$ hosted on the substrate node $n \in \mathbb{N}$, the VNF must be placed to process the flow, i.e., $x_n^f(t) = 1$. On the other hand, $x_n^f(t) = 0$.

2) *Action*: SFC embedding action space involves complicated decisions with both discrete actions (i.e., VNF placement) and continuous actions (i.e., flow scheduling) at the same time. The action $\mathcal{A}(t)$ at time slot t is expressed as

$$\mathcal{A}(t) = \left\langle x_n^f(t), r_{s(v_{n'}^{f'}, v_n^f)}(t), \forall s(v_{n'}^{f'}, v_n^f) \in \mathbb{S}_v, f \in \mathbb{F}_r, n \in \mathbb{N} \right\rangle \quad (2)$$

where $s(v_{n'}^{f'}, v_n^f) \in \mathbb{S}_v$ is the directed virtual link, $f, f' \in \mathbb{F}_r$ are two consecutive VNFs along the $r \in \mathbb{R}$, which are concluded in the set $\mathbb{F}_r$. For each $v_n^f \in \mathbb{V}$, whenever $s_{f'f} \in \mathbb{S}_r, r \in \mathbb{R}$, $r_{s(v_{n'}^{f'}, v_n^f)}$ denotes the continuous variable indicating the flow rate on $s(v_{n'}^{f'}, v_n^f) \in \mathbb{S}_v$.

The agent executes the actions if and only if the VNF placement and flow scheduling constraints are all satisfied, i.e.,

$$\begin{cases} \frac{r_n^f(t)}{\xi} \leq x_n^f(t) \leq \xi \cdot r_n^f(t); \\ T^r(t) = \sum_{s(o_r, v_n^f)} r_{s(o_r, v_n^f)}(t); \\ \sum_{s(v_{n'}^{f'}, v_n^f) \in \mathbb{S}_v} r_{s(v_{n'}^{f'}, v_n^f)}(t) = \sum_{s(v_n^{f'}, v_{n''}^{f''}) \in \mathbb{S}_v} r_{s(v_n^{f'}, v_{n''}^{f''})}(t) \end{cases} \quad (3)$$

Constraint ① represents the VNF placement constraint, where $r_n^f(t)$ represents the total traffic rate to VNF $f \in \mathbb{F}_r$ on the substrate node $n \in \mathbb{N}$ at the time slot t , ξ represents an infinite positive real number; Constraint ② represents the traffic flow routing constraint, which indicates the total egress flow amount from the origin function o_r of each service must be equal to the request rate; Constraint ③ indicates the flow balance constraint. Here, $v_n^{f'}$ and $v_n^{f''}$ represent a parent vertex and a child vertex of the $v_n^f \in \mathbb{V}$ in the VNF graph, respectively.

Note that the relationship between $r_n^f(t)$ and $r_{s(v_{n'}^{f'}, v_n^f)}(t)$ can be expressed as

$$r_n^f(t) = \sum_{s(v_{n'}^{f'}, v_n^f) \in \mathbb{S}_v} r_{s(v_{n'}^{f'}, v_n^f)}(t) \quad (4)$$

From Constraint ① in (3), it is concluded that the VNF placement action is hidden from the flow routing action. Thus, SFC embedding action space can be rewritten as

$$\mathcal{A}(t) = \left\langle r_{s(v_{n'}^{f'}, v_n^f)}(t), \forall s(v_{n'}^{f'}, v_n^f) \in \mathbb{S}_v \right\rangle \quad (5)$$

3) *Reward*: The goal of DRL agent is to maximize the cumulative rewards in a long period. The cumulative rewards can be expressed as

$$\mathcal{R}(t) = \sum_{k=1}^{\infty} \gamma^k \text{reward}_{t+k+1} \quad (6)$$

where γ represents the discount factor parameter, *reward* is the reward function.

Similar to [14], [20], we adopt the reward at each episode to guide the DRL agent to achieve the goal of minimizing the total SFC delay. It is noted that the reward at each episode is inversely proportional to the SFC delay. Therefore, with the

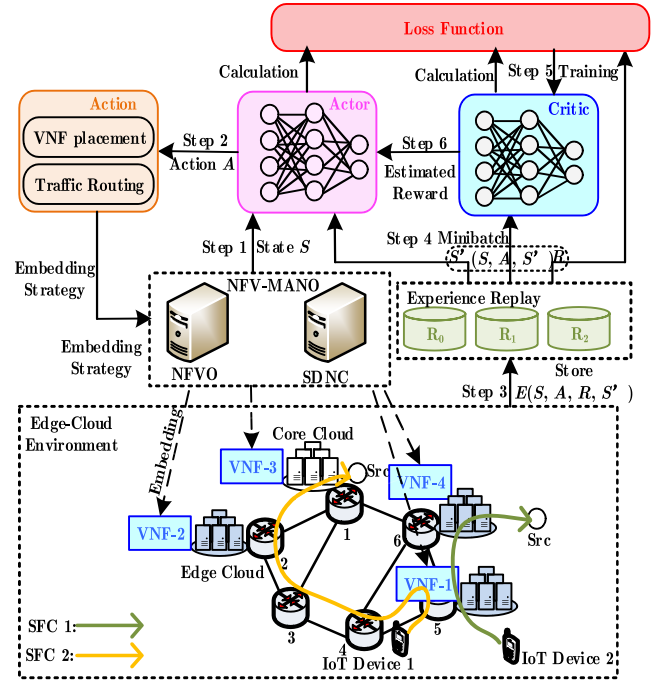


Fig. 3. Dynamic hierarchical SFC embedding scheme based on E-DDPG.

increase of episodes, cumulative rewards are negatively correlated with the metric of SFC delay. Maximizing cumulative rewards is equal to minimizing the delay of the entire SFC.

In the DRL-based SFC embedding scheme, we regard the total SFC delay as the reward of the DRL model. The reward function in the DRL framework can be mathematically defined as

$$\text{reward} = \alpha(d_{\text{total}})^{-1} = \alpha(d_{\text{proc}} + d_{\text{trans}})^{-1} \quad (7)$$

where $\alpha > 0$ is a constant parameter, d_{total} represents the total delay of an SFC that includes the VNF processing delay d_{proc} at the substrate node and transmission delay d_{trans} between this VNF and the next VNF along an SFC. More specifically, the VNF processing delay d_{proc} is generated if a VNF is activated on a server during a time slot. The entire VNF processing delay may be expressed as

$$d_{\text{proc}}(t) = \sum_{v \in \mathbb{F}_r} x_n^v(t) \cdot T_n^v, \quad \forall n \in \mathbb{N} \quad (8)$$

In the meanwhile, d_{trans} will also be deducted for the transmission delay incurred in moving the traffic flow from the servers housing its parent function. That is

$$d_{\text{trans}}(t) = \sum_{s(v_{n'}^{f'}, v_m^f) \in \mathbb{S}_v} r_{s(v_{n'}^{f'}, v_m^f)}(t) \cdot T_{l_{m,n}}, \quad \forall l_{m,n} \in \mathbb{L} \quad (9)$$

B. Framework Design

1) *Overview of the E-DDPG Framework*: The proposed E-DDPG framework is presented in Fig. 3, which includes two components, i.e., environment and agent. The main procedure can be described as follows. Firstly, actor obtains the network state $\mathcal{S}(t)$ at the current time. According to the current weights of actor, an action $\mathcal{A}(t)$ can be iteratively

calculated (**Step 1**). To increase some randomness in the exploration and obtain better exploration DDPG policy, actor also puts the noise N into the action and executes the action $\mathcal{A}(t) \sim N(\mu(\mathcal{S}(t)|\theta^\mu), \sigma^2)$, where θ^μ represents the parameters of the actor μ , σ represents the standard deviation, and N represents the action disturbance that is generated by disturbance function. After executing the action, DRL agent would acquire a reward $\mathcal{R}(t)$ from the observed environment, which enables the current state $\mathcal{S}(t)$ to transit to a new state $\mathcal{S}(t+1)$ (**Step 2**). Repeat **Step 1** and **Step 2**, the transition $E(\mathcal{S}(t), \mathcal{A}(t), \mathcal{R}(t), \mathcal{S}(t+1))$ is generated and stored in the buffer replay for training the agent (**Step 3**). We generate three types of the buffer replays including the original buffer replay $\mathbf{R}_0$, diversity buffer replay $\mathbf{R}_1$ and high-error buffer replay $\mathbf{R}_2$. When agent meets adequate transition sample, a minibatch is selected from the buffer replay for the agent training (**Step 4**). In the minibatch, there is a number of transition sample in the form of $E(\mathcal{S}(t), \mathcal{A}(t), \mathcal{R}(t), \mathcal{S}(t+1))$. Actor retrieves the batch of a new state to determine the prediction action via the actor's target network. There are four deep neural networks in the E-DDPG framework, i.e., the target networks of actor and critic, the training networks of actor and critic. **Step 5** shows the main procedure of training the deep neural networks. In the training episodes, the loss function $L(\theta^Q)$ can be calculated by the predicted actions. Critic employs the batch of a new state and the predicted actions from actor, thus determining the target Q-values by the target network of critic. In each training episode, both actor and critic get updated based on the loss function until the reward converges. The loss function can be calculated by the temporal difference, and critic is trained with the loss function (**Step 5**). In addition, critic calculates the gradients and updates them to the actor (**Step 6**).

In the SDN/NFV framework, we take into consideration the NFV management and orchestration (NFV-MANO) node, which is expected to help manage the SFC resources. Next, it is presumed that the learning agent is functioning inside the NFV-MANO node. After receiving service requests from end users, the NFV-MANO node generates different SFCs using different VNFs. At each time step, the agent of the NFV-MANO node first keeps an eye on the condition of cloud networks and end-user service requirements. Next, the system selects which VNF to install in the cloud networks before changing to a new state. The agent advances to the next time step upon obtaining an immediate reward. One VNF is allocated by an agent during the SFC embedding process for each time step. The embedding process concludes when the requested service chain's last VNF is executed. One VNF is embedded in a cloud server node for every valid choice, after which the current state is passed on to the next. Time moves forward and the next VNF of the SFC will be awarded after the current allocation of a VNF is finished. The agent will quickly learn a productive SFC embedding technique.

2) *Construction of Buffer Replay*: In order to address the slow convergence performance of the basic DDPG, two replay buffers (including the diversity buffer replay $\mathbf{R}_1$ and the high-error buffer replay $\mathbf{R}_2$) are proposed based on the original buffer replay $\mathbf{R}_0$. Firstly, the advantage of the diversity buffer replay $\mathbf{R}_1$ is that it avoids the repeated samples during the

learning process. The diversity buffer replay $\mathbf{R}_1$ adopts the bisimulation metric mechanism to measure the distance of the samples between $\mathbf{R}_0$ and $\mathbf{R}_2$. In addition, the advantage of the high-error buffer replay $\mathbf{R}_2$ is that it can guide the training agent to focus more on the valuable samples. The high-error buffer replay $\mathbf{R}_2$ sorts the selected training samples by calculating the temporal-difference (TD) error of the transition samples in $\mathbf{R}_0$, then selects the high-error samples to improve the effectivity of the subsequent parameter updating.

In the following, we present the details of two buffer replays.

(i) *High-Error Buffer Replay*

According to the learning process of the DDPG framework, the batch of the transition samples, denoted as $E(\mathcal{S}(t), \mathcal{A}(t), \mathcal{R}(t), \mathcal{S}(t+1))$, are firstly formulated and saved in the basic buffer replay $\mathbf{R}_0$ for training. In the basic DDPG framework, the uniform sampling leads to more concentration on some low value samples, which prevents the buffer replay from being efficiently utilized, and spends more time and computational resources in training the agent. The decentralized searching mechanism would result in the low training efficacy in the continuous action space. In order to search more valuable samples, we use the temporal-difference (TD) error [38] as the heuristic information, and the high-value samples are selected from the basic buffer replay $\mathbf{R}_0$ to construct the high-error buffer replay $\mathbf{R}_2$. The TD error φ is formulated as

$$\begin{aligned} \varphi(t) = & \mathcal{R}(\mathcal{S}(t), \mathcal{A}(t)) \\ & + \gamma Q'(\mathcal{S}(t+1), \mu'(\mathcal{S}(t+1)|\theta^{\mu'})|\theta^{Q'}) \\ & - Q(\mathcal{S}(t), \mathcal{A}(t)|\theta^Q) \end{aligned} \quad (10)$$

where $\mathcal{R}(\mathcal{S}(t), \mathcal{A}(t))$ denotes the reward when taking action $\mathcal{A}(t)$ in state $\mathcal{S}(t)$, $Q(\mathcal{S}(t), \mathcal{A}(t)|\theta^Q)$ represents the actor that can be used to evaluate the performance of action, $\mu(\mathcal{S}(t)|\theta^\mu)$ denotes the critic network that is used to determine the action, $Q'(\mathcal{S}(t), \mathcal{A}(t)|\theta^{Q'})$ and $\mu'(\mathcal{S}(t)|\theta^{\mu'})$ are the target actor and the target critic, respectively. Note that the sample with higher value of φ results in more effect on the value function, and the sample can be put into the high-error buffer replay $\mathbf{R}_2$.

(ii) *Diversity Buffer Replay*

$E(\mathcal{S}(t), \mathcal{A}(t), \mathcal{R}(t), \mathcal{S}(t+1))$ is selected with the probability from the high-error buffer replay $\mathbf{R}_2$ and the basic buffer replay $\mathbf{R}_0$. This would lead to the fact that there are a number of similar samples or duplicate samples, which would reduce the efficacy of the algorithm. In order to overcome this narrow, we adopt the bisimulation metric mechanism [39], [40] to expand the diversity of the transition samples $E(\mathcal{S}(t), \mathcal{A}(t), \mathcal{R}(t), \mathcal{S}(t+1))$ in the buffer replay. Before presenting the Bisimulation Metric Algorithm (BMA), we introduce the concept of the bisimulation to measure the relationship between two arbitrary states in the formulated HMDP-based model.

Definition 2: Bisimulation Relationship: $\mathcal{S}' \sim \mathcal{S}'' \in \mathbb{S}$ denotes two arbitrary states in MDP, $\mathcal{S}' \sim \mathcal{S}'' \in \mathbb{S}$ in the HMDP model can satisfy the bisimulation relationship, and two arbitrary states $\mathcal{S}' \sim \mathcal{S}'' \in \mathbb{S}$ share the same

value function and their corresponding optimal action. Assume that $\Pi \subseteq \mathbb{S} \times \mathbb{S}$ belongs to the bisimulation relationship. As for $\forall S', S'' \in \mathbb{S}$, $S' \Pi S''$ should satisfy the properties as follows:

$$\begin{cases} \textcircled{1} \forall A(t) \in \mathbb{A}, \mathcal{R}(S', A) = \mathcal{R}(S'', A); \\ \textcircled{2} \forall A(t) \in \mathbb{A}, \mathbb{H} = \mathbb{S}/\Pi, \sum_{t \in \mathbb{H}} P(t|S', A) \\ = \sum_{t \in \mathbb{H}} P(t|S'', A) \end{cases} \quad (11)$$

Here, we give the detailed theorems and the corresponding proofs regarding the bisimulation metric. It is proved that two arbitrary states in the MDP model always exist the bisimulation relationship [40], which is used to evaluate the distance between the two states. In the following, we provide the **Theorem 1**.

Theorem 1: Let $\mathbb{D}$ represent the metric set defined on the state S and $\mathcal{D} \in \mathbb{D}$. As for $\forall S', S'' \in \mathbb{S}$, define $\mathcal{G} : \mathcal{D} \rightarrow \mathcal{D}, \mathcal{G}(\mathcal{D})(S', S'') = \max_{A \in \mathbb{A}} (\mathcal{D}_A(S', S'') + \gamma T_K(\mathcal{D})(\mathcal{D}_A(S', S'')))$, where $\mathcal{D}_A(S', S'') = |\mathcal{R}(S', A) - \mathcal{R}(S'', A)|$, $0 < \gamma < 1$. Thus, $\mathcal{G} : \mathcal{D} \rightarrow \mathcal{D}$ exists a minimum fixed point $\mathcal{D}_\sim$, where $\mathcal{D}_\sim$ denotes the bisimulation metric and $\mathcal{D}_\sim(S', S'')$ is the distance between S' and S'' .

Based on **Theorem 1**, we can provide another theorem. It is proved that the similarity between samples is equal to the similarity in the corresponding value function.

Theorem 2: Assume that the distance $\mathcal{D}_\sim(S', S'') = m$, then $|Q^*(S'_1|S_1, A) - Q^*(S'_2|S_2, A)| \leq m$.

Proof: Given that S_1 and S_2 respectively represent the two arbitrary states in the HMDP model, which have the same action space. The states S_1, S_2 would transfer to another state S'_1 and S'_2 by taking a certain action $A \in \mathbb{A}$. V^* is the cumulative expected reward obtained in the state S , and Q^* is the cumulative expected reward for the state-action pair (S, A) . The difference of the cumulative reward between two arbitrary states S_1, S_2 can be represented by

$$\begin{aligned} & |Q^*(S'_1|S_1, A) - Q^*(S'_2|S_2, A)| \\ &= \left| \left(\mathcal{R}(S_1, A) + \gamma \sum_{S_1 \in \mathbb{S}} P_1(S'_1|S_1, A) V^*(S'_1) \right) - \left(\mathcal{R}(S_2, A) + \gamma \sum_{S_2 \in \mathbb{S}} P_2(S'_2|S_2, A) V^*(S'_2) \right) \right| \\ &\leq \max_{A \in \mathbb{A}} \left| \left(\mathcal{R}(S_1, A) + \gamma \sum_{S_1 \in \mathbb{S}} P_1(S'_1|S_1, A) V^*(S'_1) \right) - \left(\mathcal{R}(S_2, A) + \gamma \sum_{S_2 \in \mathbb{S}} P_2(S'_2|S_2, A) V^*(S'_2) \right) \right| \\ &= \max_{A \in \mathbb{A}} |\mathcal{D}_A(S_1, S_2) + \gamma T_K(\mathcal{D})(P_1(S_1, A, \bullet), P_2(S_2, A, \bullet))| \\ &= \mathcal{D}_\sim(S_1, S_2) = m \end{aligned} \quad (12)$$

It is concluded that $\forall S_1, S_2 \in \mathbb{S}$, $|Q^*(S'_1|S_1, A) - Q^*(S'_2|S_2, A)| \leq m$. That is to say, high-similarity training samples would result in lower value. However, low-similarity training samples could improve the efficiency of the agent training.

Considering the intervals of N episodes for the HMDP model, we adopt the BMA algorithm to calculate the distance $\mathcal{D}_\sim(S', S'')$ between the transition samples in $\mathbf{R}_0$, and the low-similarity transition samples are put into the $\mathbf{R}_1$ to ensure the diversity of the buffer replay. The pseudocode of BMA is presented in **Algorithm 1**.

3) *Parameter Updating of Critic-Actor:* The proposed E-DDPG algorithm is based on the basic actor-critic. We reconstruct the target actor-critic networks to efficiently update

Algorithm 1: BMA for Calculating the Distance

```

1 Input: Transition samples  $E(S_1, A_1, \mathcal{R}_1, S'_1)$  and  $E(S_2, A_2, \mathcal{R}_2, S'_2)$ 
2 Initialize the distance  $\mathcal{D}_\sim(S', S'') = 0$ , the replay buffers  $\mathbf{R}_0$  and  $\mathbf{R}_1$ , the discount factor  $\gamma$ , the infinitesimal positive real number  $\xi$  and the similarity distance threshold  $\lambda$ 
3 for  $K=1: \frac{\ln \xi}{\ln \gamma}$  do
4   for  $i=1: |\mathbb{A}|$  do
5      $T_K(\mathcal{D})(P_1(S_1, A, \bullet), P_2(S_2, A, \bullet))$ 
6     Calculate  $\mathcal{D}(S_1, S_2) = \max_{A \in \mathbb{A}} (\mathcal{D}_A(S_1, S_2) + \gamma T_K(\mathcal{D})(P_1(S_1, A, \bullet), P_2(S_2, A, \bullet)))$ 
7     if  $\mathcal{D}(S_1, S_2) \leq \lambda$  then
8       Put the transition sample  $E(S_1, A_1, \mathcal{R}_1, S'_1)$  into the replay buffer  $\mathbf{R}_1$ 
9 Output: Bisimulation distance  $\mathcal{D}(S_1, S_2)$ 

```

the network parameters, i.e., $Q'(S, A|\theta^{Q'})$ and $\mu'(S|\theta^{\mu'})$. The parameter updating strategies of target actor network μ and target critic network Q are computed by

$$\begin{cases} \theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \\ \theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \end{cases} \quad (13)$$

where $\theta^{Q'}$ and $\theta^{\mu'}$ respectively represent the weights of the target actor μ' and the target critic Q' , τ denotes the dynamic parameter, which can control the updating range of the target network and gradually become larger with the increase of episodes. Besides, the loss function of target actor $L(\theta^Q)$ and the gradient updating function $\nabla_{\theta^\mu} J$ can be respectively denoted as follows.

$$\begin{cases} L(\theta^Q) = E \left[((Q(s, a|\theta^Q) - r(s, a) - \gamma Q(s', \mu(s')|\theta^Q))^2 \right] \\ \nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i^N \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i} \end{cases} \quad (14)$$

where L is the training sample, γ denotes the discount reward, and J is the objective function when updating the policy. In addition, the proposed E-DDPG approach combines the noise N and the actor network $\mu(S|\theta^\mu)$ to better achieve the action exploration, which has strong randomness. When the network state is S , the process of selecting the actions by the actor network μ can be expressed by $A \sim N(\mu(S|\theta^\mu), \sigma^2)$, where σ is the intensity of strategy exploration.

4) *Formulation of the E-DDPG Algorithm:* The pseudocode of the proposed E-DDPG algorithm is summarized in **Algorithm 2**. The corresponding parameters of the actor-critic is initialized in **Lines 2-6**. More specifically, we firstly initialize the critic $Q(S, A|\theta^Q)$ and actor $\mu(S|\theta^\mu)$ with their weights θ^Q and θ^μ (**Line 3**). The weights $\theta^{Q'}$ and $\theta^{\mu'}$ can be copied by the target actor μ' and the target critic Q' (**Line 4**). Considering the diversity of samples and the high-value samples, we construct the diversity buffer replay $\mathbf{R}_1$ and the high-error buffer replay $\mathbf{R}_2$ in **Line 5**. The original buffer replay $\mathbf{R}_0$, diversity buffer replay $\mathbf{R}_1$ and high-error buffer replay $\mathbf{R}_2$ are initially set to the null set $\emptyset$ in **Line 5**, and **Line 6** initializes the bisimulation episode, time step, error

Algorithm 2: E-DDPG for Online and Dynamic SFC Embedding

1 **Input:** Substate graph $\mathbb{G} = (\mathbb{N}, \mathbb{E})$; service request $\mathbb{G}_r = (\mathbb{F}_r, \mathbb{E}_r)$; critic network $Q(\mathcal{S}, \mathcal{A}|\theta^Q)$; actor network $\mu(\mathcal{S}|\theta^\mu)$

2 **/*Initialization Phase*/**

3 Randomly initialize critic $Q(\mathcal{S}, \mathcal{A}|\theta^Q)$ and actor $\mu(\mathcal{S}|\theta^\mu)$ with their weights θ^Q and θ^μ

4 Randomly initialize the target actor μ' and the target critic Q' with their weights $\theta^{Q'}$ and $\theta^{\mu'}$

5 Initialize the original buffer replay $\mathbf{R}_0$, diversity buffer replay $\mathbf{R}_1$ and high-error buffer replay $\mathbf{R}_2$. These buffer replays are initially set to the null set $\emptyset$

6 Initialize the bisimulation episode N , time step T , error threshold ρ and similarity distance threshold λ

7 **/*Online SFC Embedding Phase*/**

8 **while** set of SFC instances SFC is not empty **do**

9 **for** totally ordered sub-SFC of $\mathbb{G}_r \in \text{SFC}$ **do**

10 **for** episode=1, ..., M **do**

11 **if** episode mod $N == 0$ **then**

12 Use the BMA algorithm to put the low similarity sample $E'(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{S}')$ into the replay buffer $\mathbf{R}_1$

13 Receive the initial observation state $\mathcal{S}_0$

14 **for** step=1, ..., T **do**

15 Select VNF placement and the traffic routing actions $\mathcal{A} \sim \mathcal{N}(\mu(\mathcal{S}|\theta^\mu), \sigma^2)$ based on the current strategy and the exploration noise;

16 Execute the action $\mathcal{S}$ and receive a reward $\mathcal{R}$, construct the new environment state $\mathcal{S}'$ and store the transition sample $E(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{S}')$ in $\mathbf{R}_0$

17 **if** $\mathbf{R}_1 \neq \emptyset$ **then**

18 Sample a random mini-batch of N transitions from $\mathbf{R}_1$ and $\mathbf{R}_2$ with probability of α and $1 - \alpha$

19 **if** $\mathbf{R}_2 \neq \emptyset$ **then**

20 Sample a random mini-batch of N transitions from $\mathbf{R}_0$ and $\mathbf{R}_2$ with probability of β and $1 - \beta$

21 **else**

22 Sample a random mini-batch of N transitions from $\mathbf{R}_0$

23 **if** $\mathcal{R}\nabla(\mathcal{S}, \mathcal{A}) + \gamma Q(\mathcal{S}', \mu(\mathcal{S}')|\theta^Q) - Q(\mathcal{S}, \mathcal{A}|\theta^Q) > \rho$ **then**

24 Store the $E(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{S}')$ in $\mathbf{R}_2$

25 Update the critic by minimizing

$$L(\theta^Q) = E \left[\left(r(\mathcal{S}, \mathcal{A}) - \gamma Q(\mathcal{S}', \mu(\mathcal{S}')|\theta^Q) \right)^2 \right]$$

26 Update the actor policy by using the sampled policy gradient $\nabla_{\theta^\mu} J$:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i^N \nabla_{\mathcal{A}} Q(\mathcal{S}, \mathcal{A}|\theta^Q)|_{\mathcal{S}=\mathcal{S}_i, \mathcal{A}=\mu(\mathcal{S}_i)} \nabla_{\theta^\mu} \mu(\mathcal{S}|\theta^\mu)|_{\mathcal{S}_i}$$

27 Update the weights of the target:

$$\begin{aligned} \theta^{Q'} &\leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \\ \theta^{\mu'} &\leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \end{aligned}$$

28 **Output:** $\langle r_{s(v_{n'}^{f'}, v_n^f)}(t), \forall s(v_{n'}^{f'}, v_n^f) \in \mathbb{S}_v \rangle, \langle x_n^f(t), \forall f \in \mathbb{F}, n \in \mathbb{N} \rangle$

threshold and similarity distance threshold. After the intervals of episodes, the BMA algorithm (Algorithm 1) is adopted to calculate the distance between the samples and accumulate the buffer replay to form the $\mathbf{R}_1$ (Line 12). In Lines 15-16, DRL agent chooses the action $\mathcal{A}$ and stores the transition $E(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{S}')$ in the original buffer replay $\mathbf{R}_0$. Lines 17-22 shows the process for selecting the buffer replay $\mathbf{R}_0$, $\mathbf{R}_1$, and $\mathbf{R}_2$, which can obtain the diversity of samples and the high-value samples for learning. In Lines 25-26, the critic networks can be iteratively updated via (14). In addition, the loss of each critic network can be calculated, and the

critic with the lowest loss is chosen to update the actor. Finally, the weights of the target networks are updated in Line 27.

5) *Algorithm Complexity Analysis:* The learning phase of E-DDPG is an offline process, which does not affect the online performance of the system. Thus, we can only analyze the running process of the E-DDPG approach. The time complexity of the E-DDPG is mainly determined by the structure of actor-critic and the size of optimized solution space. Let the number of SFC, substrate node, edge cloud and VNF be $|\mathbb{M}|$, $|\mathbb{N}|$, $|\mathbb{E}|$ and $|\mathbb{F}|$, respectively. We can divide the time complexity of

E-DDPG into two parts including the preliminary evaluation of the action execution and the iterations of action evaluation. As the input vector space consists of the traffic flow rate of $|\mathbb{M}|$ SFCs (continuous type in real) and all $|\mathbb{N}|$ VNF placement state (discrete type in binary), the number of neurons in the input layer of each neural network in actor-critic is $|\mathbb{M}| + |\mathbb{N}|$. For the output of the E-DDPG framework, the output of the actor-critic only includes the action on the traffic routing (continuous type in real), and there are $|\mathbb{N}|$ neurons in the output layer of each neural network. Here, for the state space $\mathcal{S}$, the time complexity for evaluating each action with forward propagation is $O(|\mathbb{M}| + 2|\mathbb{N}| + KL)$, where K is the number of hidden layers in actor-critic and L is the number of neurons in each hidden layer. After initially evaluating the performance of the action, E-DDPG would further evaluate the taken actions to obtain the cumulative reward by the iterative process. For each service request, E-DDPG would run the T_{max} actions to optimize the traffic flow scheduling, which runs in the computational complexity of $O(T_{max}(|\mathbb{F}| + \Gamma|\mathbb{F}|(|\mathbb{E}| + |\mathbb{N}|\log|\mathbb{E}||\mathbb{N}|)))$. In summary, the complexity of the proposed E-DDPG algorithm is $O((|\mathbb{M}| + 2|\mathbb{N}| + KL) + T_{max}(|\mathbb{F}| + \Gamma|\mathbb{F}|(|\mathbb{E}| + |\mathbb{N}|\log|\mathbb{E}||\mathbb{N}|)))$ in the running process when processing an online service request.

V. SIMULATION SETUP AND RESULTS

The simulations can be divided into two parts. Firstly, the hyper-parameters are determined, i.e., mini-batch sizes, the sample selection ratio, the thresholds of TD errors and similarity. After evaluating and determining the range for each hyper-parameter, the grid search approach [41] is used to determine the optimal hyper-parameter combination of the proposed E-DDPG algorithm. Secondly, it is noted that our literature [4] has demonstrated that the DRL-based method outperforms the traditional SFC embedding methods (i.e., heuristic-based Viterbi method [9], meta-heuristic-based GA method [10]) in the SDN/NFV-enabled cloud network. Thus, in this paper, we omit the comparisons with the traditional methods, and mainly select the most related SFC embedding algorithms (i.e., DDPG [4], A3C [4], DDQN [15], DQN [14] and forecasting-assisted heuristic method [42]) as the baseline methods.

A. Simulation Setup

To evaluate the effectivity of the proposed E-DDPG based SFC embedding algorithm in the large network scale, we conduct the simulations over a heterogeneous cloud network scenario, where a large number of IoT terminals access their own services wherever and whenever. We consider the well-known real network topology of Cogentco's Network [43] including 197 nodes and 245 links as the network topology database. Assume that there are 5 core clouds and 20 edge clouds in Cogentco, and the other substrate nodes are only used for switching the traffic. The VNFs can be host in edge/cloud servers for providing services.

In order to evaluate the utilization efficiency of substrate resources, we consider the time-varying feature of current workloads and resources quantities. More especially, we set

TABLE II
THE PARAMETER SETTINGS IN THE SIMULATION

Parameter	Minimum	Maximum
Length of SFC	2	6
Number of Edge Clouds	20	20
Number of Core Clouds	5	5
Data Transmitting Rate	10 Mb/s	50 Mb/s
IT Resources Required by Each VNF	1 Mcycle	8 Mcycle
Bandwidth Required Between VNFs	1 Mbps	5 Mbps
CPU Resources Capacity of Edge Clouds	0.2 GHz	0.3 GHz
CPU Resources Capacity of Core Clouds	2 GHz	5 GHz
Cache Resources Capacity of Switching Node	5 GB	5 GB
Initialized Packet Length of SFC Request	300 KB	500 KB
Bandwidth Capacity Between VNFs	0.2 Gbps	0.5 Gbps

computing resource of the substrate node randomly, which is between 0.2 GHz and 0.3 GHz for edge clouds, 2 GHz and 3 GHz for core clouds. The bandwidth capacity of the underlying links is set between 30 Mbps and 50 Mbps. The bandwidth capacity of the underlying links is set between 0.2 Gbps and 0.5 Gbps. The cache resource capacity of switching node is 5GB. For simulating the fluctuation of resources consumption, the resource demands of an SFC are generated by random selection from the sum of Gaussian functions with different parameters. The parameter settings in our simulations are shown in Table II. For the service request, we embed 30 different SFCs consisting of 10 types of network functions in the edge cloud networks. For each network function, we have 1 to 3 VNFs deployed in the edge servers, and 1 to 5 VNFs deployed in the core cloud servers. Note that DRL-based framework should be trained and tested in the real SDN/NFV-enabled network, but SFC is regarded as a new network technology and there is no public data set for training and testing. The amount of the flow rate corresponds the specific service request, and we adopt the trace available from [44] to mimic the time-varying traffic demands.

As for the parameter settings, we use the Adam optimization tool [16] to achieve the parameter selection for the neural network parameter, e.g., the learning rate of actor is 0.0001 and the learning rate of critic is 0.001, the discount factor of the reward $\gamma = 0.99$. The updating parameter of critic $\alpha = 0.001$ and we update the parameters $\alpha = 1.1\alpha$ every 500 episodes. The modified nonlinear unit is employed in the hidden layer of the neural network. The tanh layer is used in the output layer of critic and the variance of the Gaussian distribution is $\sigma = 0.001$. The number of episode intervals for updating the diversity buffer replay $\mathbf{R}_1$ is $N=50$ and the maximum number of steps in each episode is 2000.

The grid search is employed to select the optimal parameters for the mini-batch sizes, sample selection ratio, the thresholds of TD errors and similarity, which would be described in detail in the next subsection. In the directed graph $\mathbb{G}_r$, we have totally 50 candidate VNFs and the flow rates can be correspondingly determined. Based on the DRL-based task, the dimension of the designed state and action is analyzed as follows:

- The input network system state $\mathcal{S}(t)$ at time t is denoted by a vector consisting of the flow rate of 30 SFCs (continuous type in real) and all 120 VNF embedding

state (discrete type in binary) as $\mathcal{S}(t) = \langle T^r(t), x_n^f(t-1) \rangle, \forall f \in \mathbb{F}, n \in \mathbb{N}$. The total dimension of the state $\mathcal{S}(t)$ can be calculated as $30 + 50|\mathbb{N}|$.

- The output action $\mathcal{A}(t)$ at time t mainly consists of the action on the flow scheduling (continuous type in real) as $A(t) = \langle r_{s(v_{n'}^{f'}, v_n^f)}(t), \forall s(v_{n'}^{f'}, v_n^f) \in \mathbb{S}_v \rangle$. As for $s(v_{n'}^{f'}, v_n^f) \in \mathbb{S}_v$, it traverses the substrate link $l_{n, n'} \in \mathbb{L}$. The total dimension of the output action is $|\mathbb{L}| \cdot |\mathbb{S}_v|$.

To guarantee that the comparison between the proposed E-DDPG algorithm and the comparative algorithms is fair, all parameter settings of the basic DDPG-based SFC embedding algorithm, A3C-based SFC embedding algorithm, DDQN-based SFC embedding algorithm DQN-based SFC embedding algorithm and the forecasting-assisted heuristic solution are kept as the same in their original papers. Here, we present the main parameter settings of the compared algorithms in the simulation. The details are as follows:

1) DDPG-based SFC embedding method (“DDPG”). DDPG uses the actor-critic network and deterministic policy gradient scheme to find out the optimal action. The size of mini-batch is fixed to 128 and the learning rate is fixed to 0.001.

2) A3C-based SFC embedding method (“A3C”). Without using the experience replay, the A3C-based algorithm puts the actor-critic to the multiple threads for the synchronous training such that the service response time is minimized. The discount factor is fixed to 0.9 and entropy hyperparameter is fixed to 0.1 in A3C, repressively.

3) DDQN-based SFC embedding method (“DDQN”). DDQN determines the optimal solution from a prohibitively large solution space and then places/releases VNF Instances (VNFIs) following a threshold-based policy. The threshold of the loss function is fixed to 10^{-5} .

4) DQN-based SFC embedding method (“DQN”). DQN employs both buffer replay and target network, which can disorganize the correlation of the learning experience. The size of batch is fixed to 32 and the learning rate is fixed to 0.05.

5) Prediction-based Heuristic SFC embedding method (“PH”). In order to improve the efficacy of the SFC resource allocation, PH firstly employs the deep neural networks to predict the requested number of VNFs according to the requests, then adopt a traditional heuristic algorithm to minimize the service response time from all IoT terminals to their requested SFCs. To make such a deep neural network produce universal approximator, the number of neurons in the input, hidden and output of deep model are 33, 63 and 3, respectively.

We compare the results of the existing popular approaches with our proposal. For the traffic database, we also adopt the dataset in [44] to train the neural networks in the basic DDPG-based approach, A3C-based approach, DDQN-based approach, DQN-based approach and traditional PH approach.

All simulations were run on a machine with a Intel Core i9-9900K CPU 3.6GHz and GeForce GTX 1080 Ti GPU. The DRL-based approaches were all written in language of Python 3.6 and implemented in TensorFlow-gpu 1.4v.

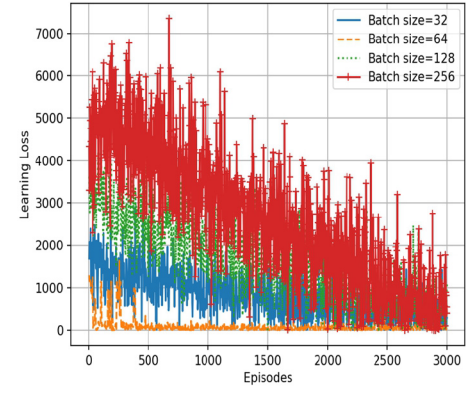


Fig. 4. The learning loss performance.

B. Results and Discussions

1) *Hyper-Parameters Selection Analysis*: Four important hyper-parameters, i.e., the mini-batch size, samples selection ratio, thresholds of TD error and similarity, are considered in the proposed E-DDPG framework. It is well-known that it is challenging to tune the hyper-parameters for the DRL-based framework. The grid search is adopted to obtain the optimal parameter combination to train the DRL agent. In the following, we first analyze the range for each parameter.

The parameter of the mini-batch size represents the numbers of the samples in each mini-batch. The mini-batch with large size would be infinitely close to the performance of full batch, and the training speed would be very slow. However, too small size would never converge in the training process. To evaluate the performance of the learning agent with different mini-batch sizes, four parameters of the mini-batch size are empirically fixed to 32, 64, 128 and 256.

Fig. 4 shows the learning loss of E-DDPG with different mini-batch sizes. The value of the learning loss is mainly determined by the mini-batch size with the increasing number of the episode. We observe that the batch size of 64 performs best, which shows that it can cause less learning loss in the training process. In contrast, the batch size of 256 is less effective and the other parameters lie in the middle of the compared ones. More specifically, the batch size of 64 can perform better than the mini-batch size of 32, 128 and 256, the reason is that lower mini-batch size can easily lead to the inapparent acceleration effect, while high mini-batch size causes the number of the weights updating. Note that the selection of the mini-batch size should not be too large or too small. We observe that the mini-batch size of 64 approaches the lowest learning loss with the increase of the episodes, which could be stabilized near the learning loss=0. The curves in Fig. 4 prove that the mini-batch size of 64 can help the proposed E-DDPG framework converge to the stabilized state.

Since the size of the mini-batch is empirically set to the n^{th} power of 2, the sample selection ratio between the diversity buffer replay R_1 and the high-error buffer replay R_2 can be divided two categories, i.e., “E-DDPG (48-16)” and “E-DDPG (16-48)”. Here, “E-DDPG (48-16)” denotes that 48 samples are selected for the diversity buffer replay R_1 in each training and 16 samples are selected for the high-error buffer replay

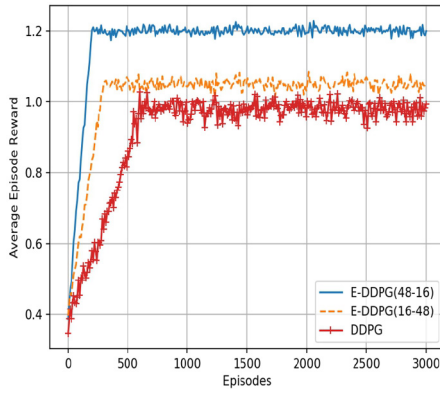


Fig. 5. The reward of sample selection ratios.

R_2 , while “E-DDPG (16-48)” denotes 16 samples for the R_1 and 48 samples for the R_2 , respectively. Fig. 5 shows the average episode reward. It can be observed that two sample selections both outperforms the basic DDPG approach, the reason is that the proposed buffer replays improve the diversity and reduce the similarity of samples, which verifies the fact that both the diversity buffer replay and the high-error buffer replay can effectively address the slow convergence problem of the basic DDPG. It can be also observed that “E-DDPG (48-16)” performs significantly better than “E-DDPG (16-48)” and DDPG. “E-DDPG (16-48)” and DDPG eventually reach around 1.21 and 1.08, respectively. “E-DDPG (48-16)”, “E-DDPG (16-48)” and DDPG become the stabilized state after approximately 270 episodes, 380 episodes and 850 episodes, respectively. The reason why “E-DDPG (48-16)” outperforms the “E-DDPG (16-48)” is because the samples in the diversity buffer replay R_1 are mainly from the high-error buffer replay R_2 , and the high-percentage number of the R_2 would result in the rapid decrease of valuable samples in R_1 . In addition, there are a number of samples that are selected for training the DRL-based model too many times. Thus, the fact that the “E-DDPG (48-16)” can significantly improve the convergence performance could be because it can select more new samples for each training.

We analyze the convergence performance under different thresholds of TD errors. It is noted that the parameter $\rho \in \{0.4, 0.5, 0.6\}$ are empirically set as the thresholds of TD error. Fig. 6 indicates the average episode reward under different TD errors. As indicated in Fig. 6, E-DDPG always performs better than the basic DDPG. This is because the high-error buffer replay R_2 contains more high-value samples that are used for agent training, thus significantly accelerating the convergence speed. Specifically, the TD error of 0.4 can rapidly obtain feasible results around 300 episodes, however, it still becomes shocked at later episodes. The reason is that the threshold is not high enough to collect fewer valuable samples into the R_2 for the agent training. Next, the TD error of 0.5 has the best reward results among the compared ones, albeit at the expense of training episodes. Although the TD error of 0.5 requires more training episode time, it could obtain the optimal solution when reaching the stabilized state. With the increase of the TD error, the TD error of 0.6 has a considerably worst

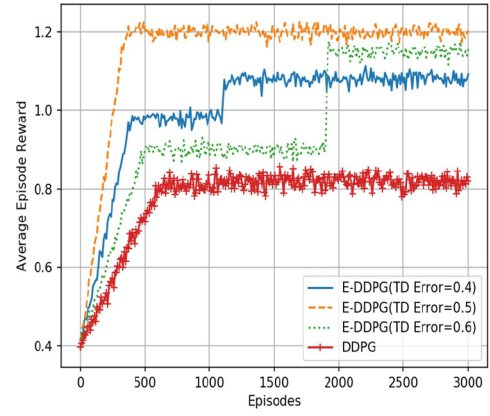


Fig. 6. The reward of different TD errors.

value of the average episode reward at the first 1800 episodes, but no worse than DDPG. This is because higher threshold of TD error might result in the fewer high-value samples, and the samples selected from the R_2 are over repeated each training time, thus endangering the convergence performance. An interesting phenomenon is that when the episodes increase to 1800, the TD error of 0.6 suddenly rises to 1.18. This is because the high-error buffer replay R_2 is gradually updated and more high valuable samples can be used for agent training with the increasing number of episodes.

Then, we discuss how E-DDPG performs with different similarity thresholds. Similar to the literature [40], we set the parameter $\lambda \in \{0.3, 0.5, 0.8\}$ as the preset thresholds on the similarity. In the simulation, we vary the parameter λ which bounds the allowed distance between states. Note that too low λ (close to 0) means that we only allow states to be aggregated when states are very close in term of the distance. Hence, the value function in the aggregated state should be very close (or identical) to the one in the original state. While $\lambda \approx 1$ means all states can be aggregated, resulting in a single-state MDP. Fig. 7 shows the average episode reward under different similarity thresholds. As shown in Fig. 7, it is observed that E-DDPG with different thresholds could significantly outperform the basic DDPG, the reason is that the reliance on the diversity buffer replay R_1 increases the average episode reward. On the other hand, we observe that the similarity of 0.5 performs significantly better than the others in term of the convergence speed and reward. For example, the similarity of 0.5 can converge around 650 episodes at the value of 1.28, the similarity of 0.3 converges around 1800 episodes at the value of 1.2 while the similarity of 0.8 converges around 2500 episodes at the value of 1.23. This is because lower value of λ can not guarantee that the high-similarity samples are selected for inserting to the R_1 , and higher value of λ results in the fact that the selected samples in the R_1 could be reused for the agent training too many times. Thus, the similarity of 0.5 can strike a good tradeoff between the high-similarity samples degeneracy problem and the sample reuse problem.

2) *Convergence Performance Analysis:* After determining the range of each parameter and evaluating the performance with different parameters in our simulation, we adopt the grid

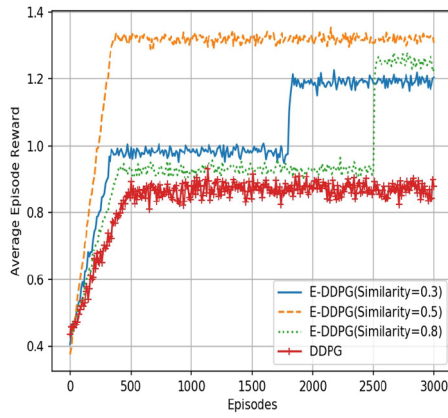


Fig. 7. The reward of different similarity thresholds.

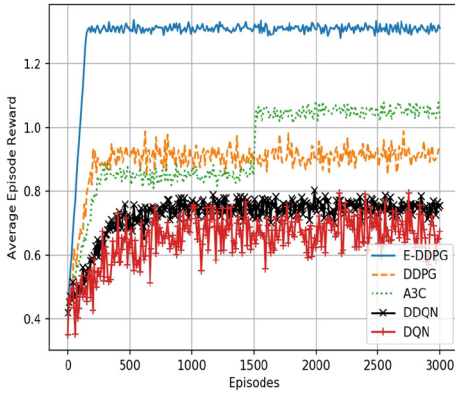


Fig. 8. The reward of different DRL-based algorithms.

search method to determine the best parameter combination for the proposed E-DDPG algorithm. The obtained parameter combination for the E-DDPG agent training can be shown as follows. The minibatch size is fixed to 64, the sample selection ratio is fixed to “48-16”, the threshold of TD error is fixed to 0.5 and the threshold of the similarity is fixed to 0.5. That is, we adopt the optimal parameter settings when applying the E-DDPG-based SFC embedding approach.

In order to validate the convergence efficiency of the proposed E-DDPG algorithm, we check the average rewards over 3000 episodes during the training process. At this stage, we did not compare with the traditional PH-based solutions since PH does not belong to the category of DRL-based framework. Fig. 8 shows the average episode reward over 3000 episodes during the training process. Every episode represents the total number of steps required to attain the desired state-action pair in order to choose the best course of action. According to the findings, the proposed E-DDPG scheme can outperform traditional benchmark schemes in terms of average episode reward. Compared to benchmark algorithms, it is observed that E-DDPG achieves convergence more quickly. From the other point of view, the training procedure of different DRL-based algorithms vary with the training episode time. DDPG, A3C and DDQN become stabilized after approximately 330 episodes, 1500 episodes and 500 episodes, the oscillations of DQN are larger upon convergence. In the simulation, we can obtain that single one training episode

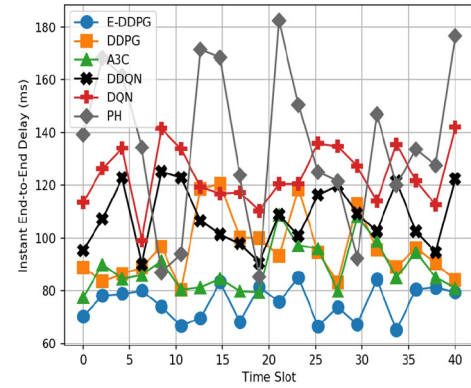


Fig. 9. The instant end-to-end delay.

time of E-DDPG is 0.353 s compared to DDPG, A3C and DDQN as 1.835 s, 0.395 s and 1.283 s, respectively. Thus, the whole training episode time of E-DDPG, DDPG, A3C and DDQN is approximately 211 s, 606 s, 593 s and 791 s, respectively. Next, DQN fails to obtain a stable average reward in the 3000 training episodes. Therefore, it is concluded that E-DDPG significantly outperforms in terms of the algorithm convergence in the agent training process, compared to DDPG, A3C, DDQN and DQN.

The fact that DDQN and DQN perform worst could be because they both adopt target strategy network and target Q network, which are only applicable for the discrete action space, thus spending much more training time in converging to a stabilized state. The other DRL-based algorithms, i.e., E-DDPG, A3C and DDPG, can achieve better solutions due to their actor-critic frameworks and policy gradient schemes. More specifically, they add a network to fit the policy function that can significantly optimize the continuous action space. We find an interesting phenomenon in term of the convergence performance and the average episode reward. It is observed that A3C converges faster and fails to obtain the better results at the initial stage, and the curve of A3C suddenly rises to the reward of 1.05 at approximately 1500 episodes. This is because A3C adopts the parallel agents for training the network that can converge faster in each training episode time. Multiple parallel A3C training may affect each other, thus resulting in worse average reward at the beginning stage. In the later episodes, the increase of data volume makes the performance of A3C better. As the increase of the data volume, A3C could gradually outperform the basic DDPG approach after approximately 1500 episodes. Ultimately, E-DDPG outperforms the compared ones, the reason is that E-DDPG employs the TD error that increases the selection probability of samples with higher value. At the same time, E-DDPG adopts the BMA algorithm to update the diversity buffer replay. This verifies the fact that the proposed E-DDPG approach achieves convergence faster as compare to DDPG, A3C, DQN and DDQN. Due to the construction of the diversity buffer replay and high-error buffer replay, E-DDPG can effectively address the slow convergence problem.

3) *Efficiency Performance Analysis:* In the following, we apply the trained agents after 3000 episodes to practically

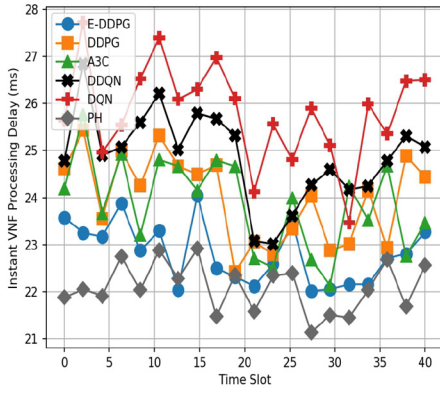


Fig. 10. The instant VNF processing delay.

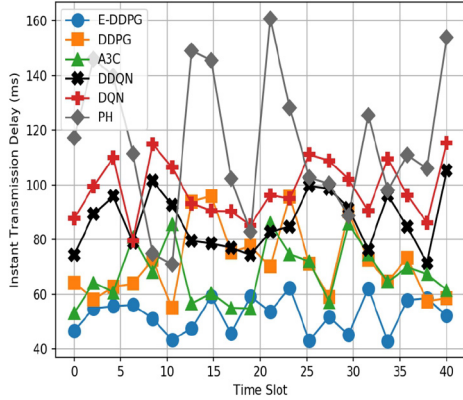


Fig. 11. The instant transmission delay.

embed SFCs to check how they perform. We first obtain the instant results during each time slot when the flow rate upper bound is set as 11 and the calculate the average results under different flow rate upper bounds for 40 time slots. The obtained simulation results are shown in Figs. 9-11. Fig. 9 shows the instant end-to-end latency of different algorithms for the SFC embedding during 40 time slots. As depicted in Fig. 9, E-DDPG can achieve the minimum end-to-end delay in most all time slots, which shows that a well-trained E-DDPG agent indeed finds better solution in minimizing the end-to-end latency. In contrast, PH performs worst in most all time slots. Besides, three traditional DRL-based approaches, i.e., DDPG, A3C, DDQN and DQN, all perform better than PH but never surpass E-DDPG. The reason why PH performs worst is because PH fails to adopt the iterative mechanism, which can not integrate new objectives and constraints, and can not approach the optimal solution in the dynamic and complex network scenarios. To indicate a clear understanding of the simulation results, we present the simulation results of the instant VNF processing delay in Fig. 10 and the instant transmission delay in Fig. 11. As shown in Fig. 10, PH could always obtain the lowest VNF processing latency, this is because PH adopts the multilayer perceptron-based model to forecast the required number of VNFs, which can determine the VNF processing resource ahead of time, and hence capture the minimum VNF processing latency. A3C and DDPG perform better for their capacity in addressing the continuous action space. However, they both perform worse

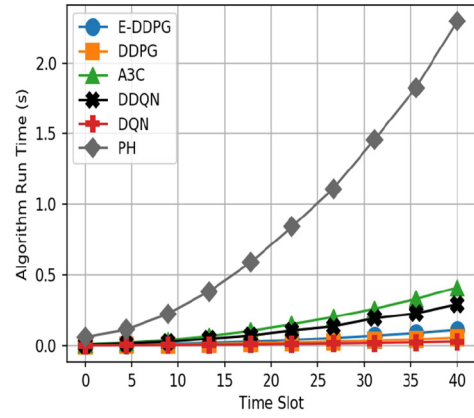


Fig. 12. The algorithm run time.

than E-DDPG due to the lack of the diversity buffer replay and high-error buffer replay. Next, DDQN performs better than DQN, this is because DDQN selects the action based on the parameters of the current Q network each time, and the target value of DDQN is lower than that of DQN. Eventually, DQN performs only slightly better than PH, the reason is that DQN can achieve a total redesign of the SFC dynamic embedding algorithm in the context where objectives and constraints are dynamic. However, it can only customize the DRL framework according to the discrete state space-based network model. When it comes to Fig. 11, we notice the phenomenon that the trends of the curves in Fig. 11 are similar to those in Fig. 9. This verifies the fact that the selection of service function path plays an important role in the performance of the end-to-end delays. Nevertheless, E-DDPG could always achieve the best SFC embedding to obtain the minimum delay in most cases.

We also investigate the results of run time and report the results in Fig. 12. It can be observed that the average run time of PH is 1.13 s, the average run time of DQN is 0.05 s, the average run time of DDPG is 0.12 s, the average run time of DDQN is 0.19 s, the average run time of A3C is 0.21 s, and the average run time of E-DDPG is 0.14 s. The results show that the average run time of PH is at least 8 times longer than that of E-DDPG, this is because PH requires a total redesign of the algorithm in the dynamic systems, which causes more computational overhead. The run time of DRL-approaches is mainly determined by the complexity of the deep structure. This verifies the fact that E-DDPG does effectively balance the tradeoff between complexity and performance.

Next, we investigate the long-term performance of all compared algorithms when the flow rate upper bound varies from 1 to 11, as reported in Figs. 13-15. In Figs. 13-15, we can obtain the average results at each flow rate upper bound and present the simulation results of the average end-to-end latency. Fig. 13 shows the average overall end-to-end delay for different flow rate upper bounds. As depicted in Fig. 13, the curve of each approach gradually becomes gentler, with the increase of flow rate upper bound. The reason is that the decreasing available IT resources and bandwidth might result in more rejected traffic requests. It is observed that E-DDPG can outperform the compared ones in most cases, which shows that E-DDPG can well guide the central controller to embed

the SFCs. Here, we notice an interesting phenomenon that PH can only outperform the compared ones for the first few flow rate upper bounds, however, PH gradually loses their position for the next flow rate upper bounds. The reason is that PH can well approach the near-optimal solution at the small-data network scenarios, where the flow rate upper bound is very small. As the increasing number of the flow rate upper bounds, PH may have a rapid convergence at the sacrifice of trapping a local minimum delay, which is not effective in the dynamic network scenarios. As a traditional heuristic method, PH fails to obtain the optimal solution when the network problem's scale becomes larger and can not follow the increasing trend of the other compared algorithms does. In addition, PH requires a total redesign of SFC embedding algorithm in the context where objectives and constraints are dynamic. Since the DRL-based methods, i.e., E-DDPG, DDPG, A3C, DDQN and DQN, have advantage of dealing with SFC embedding in the dynamics, and they can obtain better solutions than the traditional methods in such an edge-cloud network scenario. That is also confirmed that the DRL-based methods can efficiently deal with SFC embedding. In particular, we can observe that DDQN and DQN performs worse than DDPG and A3C. The reason is that DDQN and DQN could only handle discrete and low-dimensional action spaces, which are not appropriate for large-scale action space. As for continuous action space-based modelling, which involves the complicated decisions and the continuous actions at the same time, this might further make DDQN and DQN hinder the convergence of the agent training. In contrast, both DDPG and A3C employ the actor-critic and policy gradient scheme, which could operate over the continuous action spaces. Finally, we find another interesting phenomenon in Fig. 13 that when the flow rate upper bound is less than 6, A3C performs slightly worse than DDPG, the reason is that A3C performs inefficiently and causes the high variance when evaluating a policy in the small and medium network scenarios. A3C causes high variance, since the value depends on a chain of actions that might lead to a number of different states. When the flow rate upper bound is more than 6, A3C can significantly outperform DDPG but never surpass E-DDPG. This confirms that the superiority of A3C gradually emerges as the increase scale of network problem, which is the advantage of parallel scheme. However, A3C still suffers from the “curse of dimensionality”, which easily traps into the suboptimal solution when the problem's scale becomes larger.

We also present the specific simulation results of the VNF processing delay in Fig. 14 and the transmission delay in Fig. 15. As shown in Fig. 14, we can observe that E-DDPG, DDPG, A3C and DQN obtain the similar trends all the 11 flow rate upper bounds, while PH always gives the lowest average VNF processing latency, this is because PH combines online deep learning model for VNF resource prediction and offline SFC dynamic resource allocation, which determines the VNF resource requirements ahead of time, and hence significantly reduce the VNF processing time. Fig. 15 shows the similar performance of the total average end-to-end delays as Fig. 13 has shown. The explanation is also same, and thus omitted here. Through these results, it comes to a conclusion that E-DDPG can accelerate the agent training and almost

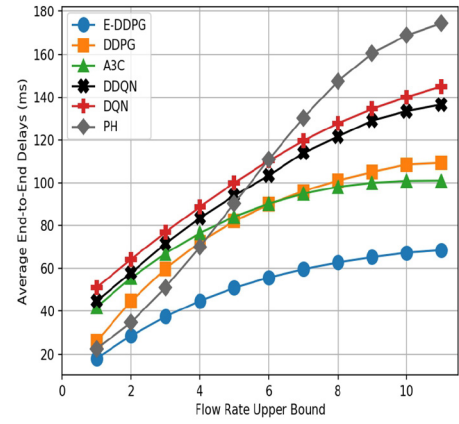


Fig. 13. The average end-to-end delay.

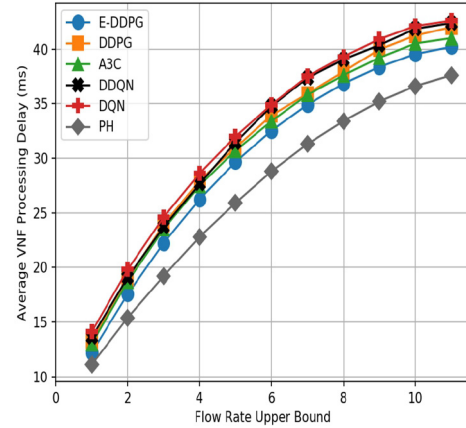


Fig. 14. The average VNF processing delay.

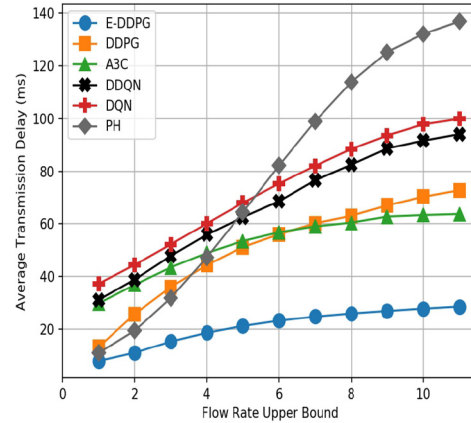


Fig. 15. The average transmission delay.

perform better than the compared algorithms, which confirms that the favorable performance of our proposal applying in the SDN/NFV-enabled networks. It is proved that the proposed E-DDPG framework can be more appropriate for the dynamic cloud networks.

VI. CONCLUSION

In this article, we propose an E-DDPG-based SFC dynamic embedding algorithm for SDN/NFV-enabled networks. This

article is motivated by the fact that the traditional solutions (e.g., exact, heuristic, meta-heuristic and game) are subject to the complexity of the cloud network scenarios, and the current popular DRL-based approaches (e.g., DQN, DDQN, DDPG and A3C) easily suffer from the problem of “curse of dimension”. Thus, we improve the newly DDPG algorithm with recent advances in DRL, to present a practical solution that has been proved completely appropriate for the dynamic and complex cloud networks. The obtained simulation results show that the proposed E-DDPG algorithm can improve the efficacy of the training convergence, and outperform in terms of the end-to-end delay compared with the basic DDPG, A3C, DDQN, DQN and the traditional heuristic-based algorithm.

In the future, we will continuously improve the convergence of the agent training. Furthermore, we will also focus on addressing how to fill the gap between the simulator and the realistic platforms for the E-DDPG-based SFC embedding algorithm implementation, and consider implementing our proposal in the realistic OpenDaylight/Openstack platforms.

ACKNOWLEDGMENT

The authors would like to thank the editor and anonymous reviewers for the instructive comments, and we appreciate your warm work earnestly.

REFERENCES

- [1] *Network Functions Virtualization (NFV); Use Cases*, Eur. Telecommun. Stand. Inst., Sophia Antipolis, France, Oct. 2013. [Online] Available: http://www.etsi.org/deliver/etsi_gs/nfv/001_099/001/01.01.01_60/gs_nfv001v010101p.pdf
- [2] Y. Xiao et al., “NFVdeep: Adaptive online service function chain deployment with deep reinforcement learning,” in *Proc. IEEE/ACM 27th Int. Symp. Qual. Service (IWQoS)*, Phoenix, AZ, USA, 2019, pp. 1–10, doi: [10.1145/3326285.3329056](https://doi.org/10.1145/3326285.3329056).
- [3] “Network functions virtualization (NFV) white paper #3,” Eur. Telecommun. Stand. Inst., Sophia Antipolis, France, Oct. 2014. [Online] Available: http://portal.etsi.org/NFV/NFV_White_Paper3.pdf
- [4] Y. Liu, Y. Lu, X. Li, W. Qiao, Z. Li, and D. Zhao, “SFC embedding meets machine learning: Deep reinforcement learning approaches,” *IEEE Commun. Lett.*, vol. 25, no. 6, pp. 1926–1930, Jun. 2021, doi: [10.1109/LCOMM.2021.3061991](https://doi.org/10.1109/LCOMM.2021.3061991).
- [5] Y. Liu et al., “Reliability-aware service chaining mapping in NFV-enabled networks,” *ETRI J.*, vol. 42, no. 2, pp. 207–233, 2019.
- [6] S. Yang, F. Li, S. Trajanovski, X. Chen, Y. Wang, and X. Fu, “Delay-aware virtual network function placement and routing in edge clouds,” *IEEE Trans. Mobile Comput.*, vol. 20, no. 2, pp. 445–459, Feb. 2021.
- [7] R. Mijumbi, J. Serrat, J.-L. Gorricho, N. Bouten, F. De Turck, and S. Davy, “Design and evaluation of algorithms for mapping and scheduling of virtual network functions,” in *Proc. 1st IEEE Conf. Netw. Softwarization (NetSoft)*, London, U.K., 2015, pp. 1–9.
- [8] Y. Liu, Y. Lu, W. Qiao, and X. Chen, “A dynamic composition mechanism of security service chaining oriented to SDN/NFV-enabled networks,” *IEEE Access*, vol. 6, pp. 53918–53929, 2018.
- [9] F. Bari, S. R. Chowdhury, R. Ahmed, R. Boutaba, and O. C. M. Bandeira Duarte, “Orchestrating virtual network functions,” *IEEE Trans. Netw. Service Manag.*, vol. 13, no. 4, pp. 725–739, Dec. 2016.
- [10] W. Rankothge, F. Le, A. Russo, and J. Lobo, “Optimizing resource allocation for virtualized network functions in a cloud center using genetic algorithms,” *IEEE Trans. Netw. Service Manag.*, vol. 14, no. 2, pp. 343–356, Jun. 2017.
- [11] M. Luizelli, W. L. C. Cordeiro, L. S. Buriol, and L. P. Gaspary, “A fix-and-optimize approach for efficient and large scale virtual network function placement and chaining,” *Comput. Commun.*, vol. 102, pp. 67–77, Apr. 2016.
- [12] A. Abujoda and P. Papadimitriou, “DistNSE: Distributed network service embedding across multiple providers,” in *Proc. 8th COMSNETS*, 2016, pp. 1–8.
- [13] M. Dieye, W. Jaafar, H. Elbiaze, and R. H. Glitho, “Market driven multidomain network service orchestration in 5G networks,” *IEEE J. Sel. Areas Commun.*, vol. 38, no. 7, pp. 1417–1431, Jul. 2020.
- [14] X. Fu, F. R. Yu, J. Wang, Q. Qi, and J. Liao, “Dynamic service function chain embedding for NFV-enabled IoT: A deep reinforcement learning approach,” *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 507–519, Jan. 2020.
- [15] J. Pei, P. Hong, M. Pan, J. Liu, and J. Zhou, “Optimal VNF placement via deep reinforcement learning in SDN/NFV-enabled networks,” *IEEE J. Sel. Areas Commun.*, vol. 38, no. 2, pp. 263–278, Feb. 2020.
- [16] P. T. A. Quang, Y. Hadjadj-Aoul, and A. Outgarts, “A deep reinforcement learning approach for VNF forwarding graph embedding,” *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 4, pp. 1318–1331, Dec. 2019.
- [17] D. T. Nguyen, C. Pham, K. K. Nguyen, and M. Cheriet, “Placement and chaining for run-time IoT service deployment in edge-cloud,” *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 1, pp. 459–472, Mar. 2020.
- [18] Y. Liu, H. Lu, X. Li, D. Zhao, W. Wu, and G. Lu, “A novel approach for service function chain dynamic orchestration in edge clouds,” *IEEE Commun. Lett.*, vol. 24, no. 10, pp. 2231–2235, Oct. 2020.
- [19] W. Qiao, Y. Liu, Y. Lu, X. Li, J. Yan, and Z. Yao, “A novel approach for service function chain embedding in cloud datacenter networks,” *IEEE Commun. Lett.*, vol. 25, no. 4, pp. 1134–1138, Apr. 2021, doi: [10.1109/LCOMM.2020.3043845](https://doi.org/10.1109/LCOMM.2020.3043845).
- [20] X. Fu, F. R. Yu, J. Wang, Q. Qi, and J. Liao, “Service function chain embedding for NFV-enabled IoT based on deep reinforcement learning,” *IEEE Commun. Mag.*, vol. 57, no. 11, pp. 102–108, Nov. 2019.
- [21] B. Gao, Z. Zhou, F. Liu, and F. Xu, “Winning at the starting line: Joint network selection and service placement for mobile edge computing,” in *Proc. INFOCOM IEEE Conf. Comput. Commun.*, Paris, France, 2019, pp. 1459–1467, doi: [10.1109/INFOCOM.2019.8737543](https://doi.org/10.1109/INFOCOM.2019.8737543).
- [22] X. Fei, F. Liu, Q. Zhang, H. Jin, and H. Hu, “Paving the way for NFV acceleration: A survey and future directions,” *ACM Comput. Surveys*, vol. 53, no. 4, pp. 1–42, 2020, doi: [10.1145/3397022](https://doi.org/10.1145/3397022).
- [23] J. G. Herrera and J. F. Botero, “Resource allocation in NFV: A comprehensive survey,” *IEEE Trans. Netw. Service Manag.*, vol. 13, no. 3, pp. 518–532, Sep. 2016.
- [24] R. Mijumbi, J. Serrat, J.-L. Gorricho, N. Bouten, F. De Turck, and R. Boutaba, “Network function virtualization: State-of-the-art and research challenges,” *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 236–262, 1st Quart., 2016.
- [25] D. Chemodanov, P. Calyam, and F. Esposito, “A near optimal reliable composition approach for geo-distributed latency-sensitive service chains,” in *Proc. INFOCOM IEEE Conf. Comput. Commun.*, Paris, France, 2019, pp. 1792–1800.
- [26] D. Bhamare, M. Samaka, A. Erbad, R. Jain, L. Gupta, and H. A. Chan, “Multi-objective scheduling of micro-services for optimal service function chains,” in *Proc. IEEE Int. Conf. Commun. (ICC)*, Paris, France, 2017, pp. 1–6.
- [27] V. Eramo, M. Ammar, and F. G. Lavacca, “Migration energy aware reconfigurations of virtual network function instances in NFV architectures,” *IEEE Access*, vol. 5, pp. 4927–4938, 2017.
- [28] M. M. Tajiki, S. Salsano, L. Chiaraviglio, M. Shojafar, and B. Akbari, “Joint energy efficient and QoS-aware path allocation and VNF placement for service function chaining,” *IEEE Trans. Netw. Service Manag.*, vol. 16, no. 1, pp. 374–388, Mar. 2018.
- [29] L. Qu, M. Khabbaz, and C. Assi, “Reliability-aware service chaining in carrier-grade softwarized networks,” *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 558–573, Mar. 2018.
- [30] A. Shamel-Sendi, Y. Jarraya, M. Pourzandi, and M. Cheriet, “Efficient provisioning of security service function chaining using network security defense patterns,” *IEEE Trans. Services Comput.*, vol. 12, no. 4, pp. 534–549, Aug. 2019.
- [31] Z. Xu, F. Liu, T. Wang, and H. Xu, “Demystifying the energy efficiency of network function virtualization,” in *Proc. IEEE/ACM 24th Int. Symp. Qual. Service (IWQoS)*, Beijing, China, 2016, pp. 1–10, doi: [10.1109/IWQoS.2016.7590429](https://doi.org/10.1109/IWQoS.2016.7590429).
- [32] R. Lin, L. He, S. Luo, and M. Zukerman, “Energy-aware service function chaining embedding in NFV networks,” *IEEE Trans. Services Comput.*, vol. 16, no. 2, pp. 1158–1171, Apr. 2023, doi: [10.1109/TSC.2022.3162328](https://doi.org/10.1109/TSC.2022.3162328).
- [33] X. Fei, F. Liu, H. Xu, and H. Jin, “Towards load-balanced VNF assignment in geo-distributed NFV infrastructure,” in *Proc. IEEE/ACM 25th Int. Symp. Qual. Service (IWQoS)*, 2017, pp. 1–10, doi: [10.1109/IWQoS.2017.7969166](https://doi.org/10.1109/IWQoS.2017.7969166).

- [34] T. Wang, H. Xu, and F. Liu, "Multi-resource load balancing for virtual network functions," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Atlanta, GA, USA, 2017, pp. 1322–1332, doi: [10.1109/ICDCS.2017.233](https://doi.org/10.1109/ICDCS.2017.233).
- [35] B. Addis, M. Gao, and G. Carello, "On the complexity of a virtual network function placement and routing problem," *Electron. Notes Discrete Math.*, vol. 69, pp. 197–204, Aug. 2018.
- [36] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [37] S. Singh, A. Okun, and A. Jackson, "Learning to play Go from scratch," *Nature*, vol. 550, no. 10, pp. 336–337, 2017.
- [38] G. Stampa, M. Arias, D. Sánchez-Charles, V. Muntés-Mulero, and A. Cabellos, "A deep-reinforcement learning approach for software defined networking routing optimization," in *Proc. ACM CoNEXT*, 2017, pp. 1–3.
- [39] R. Givan, T. Dean, and M. Greig, "Equivalence notions and model minimization in Markov decision processes," *Artif. Intell.*, vol. 147, pp. 163–223, Jul. 2003.
- [40] N. Ferns, P. Panangaden, and D. Precup, "Metrics for finite Markov decision processes," in *Proc. 20th Conf. Uncertainty Artif. Intell.*, 2004, pp. 162–169.
- [41] P. Bhat, H. Prosper, S. Sekmen, and C. Stewart, "Optimizing event selection with the random grid search," *Comput. Phys. Commun.*, vol. 228, pp. 245–257, Jul. 2018.
- [42] J. Zhang, Y. Liu, Z. Li, and Y. Lu, "Forecast-assisted service function chain dynamic deployment for SDN/NFV-enabled cloud management systems," *IEEE Syst. J.*, vol. 17, no. 3, pp. 4371–4382, Sep. 2023, doi: [10.1109/JSYST.2023.3263865](https://doi.org/10.1109/JSYST.2023.3263865).
- [43] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan, "The Internet topology zoo," *IEEE J. Sel. Areas Commun.*, vol. 29, no. 9, pp. 1765–1775, Oct. 2011.
- [44] "Brain flow trace." Accessed: Jan. 20, 2021. [Online]. Available: <https://www.zib.de/itds/brain>



Yicen Liu received the Ph.D. degrees in communication and information system from Research Institute of Next Generation Network Technology, AEU, Shijiazhuang, China, in 2021.

He is currently an Assistant Researcher with the National Key Laboratory of Blind Signal Processing, Chengdu, China. He has published over 30 articles in refereed conferences and journals in the above areas. His current interests are key technologies of 6G communication, SDN/NFV/AI technology, and network slicing. He has been invited to become a Reviewer of some top journals, such as IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE INTERNET OF THINGS JOURNAL, and IEEE COMMUNICATIONS LETTERS AND COMPUTER NETWORKS (Elsevier).



Junning Zhang received the B.E. degree in vehicle engineering from Southwest Jiaotong University, Chengdu, China, in 2014, the M.E. and Ph.D. degrees in electronic science and technology from the Army Engineering University of PLA, Shijiazhuang, China, in 2016 and 2020, respectively. He is currently a Lecturer with the Institute of Electronic Countermeasure, National University of Defense Technology, Hefei, China. He has published over 30 articles in refereed conferences and journals in the above areas. His research interests include

NFV, deep learning, intelligent electromagnetic environment perception, signal processing, SLAM, computer vision, and pattern recognition.